A Connected CSS Absorbing Set Without Gallager-B Trapping

October 26, 2026

Abstract

We construct a fifteen-qubit CSS code with one encoded qubit and a $(7, 0)$-absorbing set on which every supported binary Z-error is corrected by zero-initialized syndrome-based Gallager-B at initialization or after one iteration. Both the hard-decision syndrome and outgoing extrinsic-message parities accept the result. The actual decoding Tanner graph is connected, its fifteen checks are distinct, and the stabilizer space admits no direct product decomposition across any nontrivial partition of the physical qubits. Thus the successful absorbing support is not an isolated stabilizer-state factor. An explicit overlap spanning tree certifies the last assertion. The complete finite construction, graph connectivity, recovery modulo the whole opposite stabilizer span and unbounded stopped-execution properties are checked in Lean, and independently reconstructed from literal matrices.

1 Result and motivating question

Morris, Pllaha and Kelley ask in Section VI of *Absorbing Sets in Quantum LDPC Codes*, arXiv:2307.14532v2, May 10, 2024, whether every absorbing set is a trapping set for their syndrome-based Gallager-B decoder [1]. Their Definition 5 compares even and odd induced check neighbors. Definition 3 describes persistent syndrome mismatch or nonconvergence for some error supported on the set. The simplex matrix already supplies an absorbing component with no such failure; a disconnected CSS embedding can also make all its supported errors correctable modulo stabilizers [2]. This motivates a stronger, project-derived question: can both properties hold in a connected code without an isolated stabilizer-state factor?

Theorem 1. *There is a binary CSS pair* (H_X, H_Z) *on fifteen physical qubits with* $\operatorname{rank} H_X = \operatorname{rank} H_Z = 7$ *and the following properties.*

1. *The actual* H_X *Tanner graph, including all fifteen distinct check vertices and all fifteen variable vertices, is connected.*
2. *The first seven variables form a* $(7, 0)$*-absorbing set* A*.*
3. *Every* Z*-error supported on* A *is accepted at initialization or within one full Gallager-B iteration, under both hard-decision and outgoing extrinsic-edge syndrome tests. Its residual belongs to* $\operatorname{row} H_Z$*.*
4. *The binary stabilizer space has no product decomposition across any nonempty proper partition of the fifteen physical coordinates.*

In particular the code encodes one qubit, and A *is not trapping for the specified stopped decoder.*

The matrices and decoder are given in Sections 2 and 3; connectivity and the product obstruction are proved in Section 4. Appendix A gives the exact formal model, complete central proof bodies and current and immutable proof links. The complete source comparison and access ledger accompanies the artifact.

The construction is asymmetric: the matrix detecting Z-errors is H_X, while successful recovery is measured modulo the entire span of H_Z. It permits redundant checks and four-cycles; the exterior auxiliary variable C has degree one in the decoding graph. It asserts neither correction of both Pauli sectors nor a high-girth or degree-three counterexample. These qualifications determine the mathematical statement.

2 Literal CSS matrices

Let H be the 7×7 matrix with $H_{a,v} = a \cdot v$, for nonzero $a, v \in \mathbb{F}_2^3$ in integer order $1, \ldots, 7$, least significant binary digit first. Its rows, as integer bit masks, are

$$(85, 102, 51, 120, 45, 30, 75).$$

Let G consist of rows $1, 2, 4$ of H, set $K = \binom{G}{\mathbf{1}}$, $u = H\mathbf{e}_1^{\mathsf{T}}$ and $v = G\mathbf{e}_1^{\mathsf{T}}$. Order the physical coordinates as $A_1, \ldots, A_7, B_1, \ldots, B_7, C$. The actual matrices are

$$H_X = \begin{pmatrix} H & u\mathbf{e}_1 & 0 \\ 0 & H & 0 \\ 0 & \mathbf{e}_1 & 1 \end{pmatrix}, \qquad H_Z = \begin{pmatrix} K & 0 & 0 \\ v\mathbf{e}_1 & G & v \end{pmatrix}. \tag{1}$$

Here $\mathbf{e}_1$ is a row vector of length seven; block zero sizes are determined by the displayed matrices. H_X has fifteen rows and H_Z has seven rows. For completely unambiguous reproduction, their row masks with coordinate A_1 as the least significant bit are

$$\begin{aligned} H_X: \quad & (213, 102, 179, 120, 173, 30, 203, \\ & \qquad 10880, 13056, 6528, 15360, 5760, 3840, 9600, 16512), \\ H_Z: \quad & (85, 102, 120, 127, 27265, 13056, 15360). \end{aligned}$$

They can also be obtained from the earlier two-block CSS pair, adjoining an X-stabilized qubit C, by CNOTs $A_1 \to B_1$ and $C \to B_1$. The exact binary gate transformations are checked by `two_CNOT_X_bridge` and `two_CNOT_Z_bridge`; our claims are checked directly on the resulting matrices rather than inferred from an unspecified equivalence.

Lemma 2. *The matrices in* (1) *commute as CSS checks and both have rank seven. All fifteen X rows are distinct. The set A is* $(7, 0)$*-absorbing.*

Proof. The dot product of each of the fifteen displayed X rows with each of the seven Z rows is zero. A basis for the X rows consists of masks

$$(213, 102, 120, 10880, 13056, 15360, 16512).$$

Each actual X row is a combination of these seven actual rows, and all 128 combinations of the basis are distinct. The seven Z rows likewise have 128 distinct combinations. These are finite proofs of both ranks; they do not infer independence from merely having seven generators. The distinctness test compares all pairs of actual X rows.

The first seven checks restrict on A to H, with induced degree four; the remaining checks have no neighbor in A. Each variable of A has four even induced neighbors and no odd neighbor. Thus A has seven variables and zero odd checks, and satisfies the strict absorbing inequality. All assertions are checked on the literal data by the declarations in the proof map of Appendix A. □

The CSS dimension formula now gives $k = 15 - 7 - 7 = 1$. Explicitly, Z mask 32641 commutes with every X check and is outside $\operatorname{row} H_Z$. X mask 16256 commutes with every Z check and anticommutes with that Z word. These witnesses are checked in the same model.

3 Decoder and all supported errors

All messages are binary. Variable-to-check messages start at zero. A check sends its input syndrome bit plus the XOR of its other incoming messages. A variable sends a strict majority of its other incoming check messages, with a tie returning zero; its hard decision uses all its check neighbors. These are the zero-initialized, extrinsic, zero-tie rules of Section II-B of [1]. We test both the syndrome of the hard word and the parity of the outgoing variable messages at each check.

Lemma 3. *For every error $e \in \mathbb{F}_2^7$ on A, the first hard output is $(\mathbf{e}_p, 0, 0)$ if $p = \sum_{i=1}^{7} e_i i \neq 0$ in $\mathbb{F}_2^3$, and zero otherwise. Its syndrome and the first outgoing-edge syndrome both equal the input syndrome. Its residual is in* $\operatorname{row} H_Z$.

Proof. The first seven input syndrome bits are $a \cdot p$; the other eight are zero. At A_i, the initial one-vote count is zero for $p = 0$, four for $p = i$, and two otherwise. This follows by counting solutions of the two binary equations $a \cdot i = a \cdot p = 1$. Consequently the hard output on A is as stated. The target B_1 has nine neighbors with at most four one-votes, so its hard output is zero. Every other exterior variable receives only zeros.

The extra check involving B_1 and C is decisive for the extrinsic test: even after excluding a zero vote, B_1 sees at most four ones among eight votes and sends zero. Every exterior outgoing message is therefore zero. On A, variable p sends one on every incident edge. When $i \neq p$, excluding a zero from two ones and two zeros sends one; excluding a one sends zero. A check with syndrome one receives exactly one one from A, and a check with syndrome zero receives either zero or four ones. Its outgoing parity is therefore the input syndrome in both cases.

For the actual matrices, complete enumeration of $e \in \mathrm{BitVec}(7)$ proves the hard-output formula, both syndrome equalities, and the existence of seven binary coefficients expressing $e + \hat{e}$ in the *whole* H_Z span. These are the Lean theorems `first_output_formula`, `first_hard_matches`, `first_edge_matches` and `complete_quantum_recovery`. The independent verifier reconstructs all 128 inputs with literal lists and independently enumerates the complete stabilizer spans. Thus the quantum conclusion does not confuse a zero residual syndrome with stabilizer equivalence. □

Zero syndromes are accepted at initialization; otherwise Lemma 3 gives acceptance at the first step. A halted decoder is extended by its constant terminal output. For every input, every check is then correct and every variable constant at all times $t \geq 1$. The structural Lean proofs `all_checks_eventually_correct` and `all_variables_converge` imply `selected_not_trapping`. This is an infinite-time stopped-execution statement, not a finite observation of unhalted message dynamics.

4 Connectivity and intrinsic physical coupling

Lemma 4. *The actual thirty-vertex bipartite decoding graph is connected.*

Proof. Every physical variable shares an actual X check with B_1. Every check has at least one variable neighbor. Hence every variable has a walk of length at most two to B_1, and every check has a walk of length at most three. Reverse and concatenate these walks to connect any ordered pair

of graph vertices. `decoding_graph_connected` constructs these walks with the literal adjacency relation. □

Connectivity of a chosen redundant check representation alone does not exclude a product stabilizer code. We therefore prove a representation independent obstruction. Write $S = (\operatorname{row} H_X) \times (\operatorname{row} H_Z)$ for the binary stabilizer space. For a cut Q, let r_Q restrict both binary components to Q, zeroing the other coordinates. A direct product across $Q \mid Q^c$ requires $r_Q(S) \subseteq S$. We use this necessary closure property as the exact binary meaning of a physical product split in the formal theorem.

Lemma 5. *For every nonempty proper $Q \subset \{0, \ldots, 14\}$, there are actual X and Z rows whose restrictions to Q anticommute. In particular $r_Q(S) \not\subseteq S$.*

Proof. Table 1 gives fourteen pairs of rows whose support intersections are exactly the two coordinates in the first column. Those pairs form a spanning tree on all fifteen physical coordinates. A proper nonempty cut must cross an edge: if no edge crossed, equality of cut membership would propagate from vertex zero along the tree to every vertex, making the cut empty or full. At a crossed pair, the restricted X/Z dot product is one. If the projected X row belonged to $\operatorname{row} H_X$, it would commute with every Z row, a contradiction. Thus projecting the corresponding pure X stabilizer exits S.

Lean checks each row-pair projection identity and proves the tree-cut argument by explicit Boolean propagation. `cut_anticommutation` and `no_physical_product_cut` quantify over all fifteen-bit cuts. No assumption about a preferred independent generating set is used. □

Edge	X row	Z row	Edge	X row	Z row
$0, 7$	0	4	$7, 14$	14	4
$0, 4$	2	0	$4, 5$	2	2
$0, 2$	4	0	$4, 3$	5	2
$0, 6$	6	0	$2, 1$	5	1
$7, 11$	9	4	$11, 12$	9	6
$7, 9$	11	4	$11, 10$	12	6
$7, 13$	13	4	$9, 8$	12	5

Table 1: Overlap spanning tree. All coordinate and row indices in this table are zero-based, matching the Lean file.

Lemmas 2–5 prove Theorem 1. The product obstruction concerns the given physical tensor factors; it does not assert an obstruction to global Clifford decoding.

5 Source interpretation and earlier ingredients

Hamming syndrome representatives and the simplex/Steane CSS ingredients are classical [3, 4]. Local anticommutation of restricted stabilizers is also a standard tool in stabilizer entanglement [5]. A modern statement for stabilizer subspaces is Lemma 1 of Makuta and Augusiak [6]. We use the binary obstruction directly and claim no new general entanglement criterion. The concrete contribution is its coexistence with absorption and complete successful Gallager-B recovery in a connected actual decoding graph.

The inspected source's dynamical Definition 3 and quantum logical failure must be distinguished. Here both dynamical success and recovery modulo actual opposite Z stabilizers hold. The

source also discusses logical miscorrections and uses a row-space expression involving its H_A in Theorem 5. Our asymmetric pair does not settle an interpretation that requires that very decoded matrix to generate the error-type stabilizers, or requires $H_X = H_Z$. We assert the explicit theorem above rather than certify every possible reading of the author's blanket question. The positive-b result in the source has different hypotheses; our example has $b = 0$. The zero-tie and stopping conventions are essential specifications.

The exact question inspected is arXiv v2, Sections II-B and VI and Definitions 3 and 5. Journal metadata gives March 3, 2026 publication, but the journal full text remained subscription restricted. The accompanying source ledger records this version gap and the scope of the prior-art comparison.

Source	Relevant dated milestone	Days to October 26, 2026
Hamming	April 1950 publication	About 76.5 years
Steane	January 29, 1996 preprint	11,228
Fattal et al.	June 23, 2004 preprint	8,160
Morris et al.	July 26, 2023 first preprint	1,188
Inspected question	May 10, 2024 arXiv v2	899
Makuta–Augusiak	December 14, 2023 preprint	1,047
Inspected main article	August 23, 2025 journal	429
Morris et al. journal	March 3, 2026 metadata	237
This paper	October 26, 2026	0

Table 2: Reference chronology, with day precision except Hamming (month precision). Ingredient age is not the time an open question remained unsolved.

6 Conclusion

A connected ambient decoding graph and absence of every physical product factor do not force this $(7, 0)$-absorbing support to fail Gallager-B. Every supported Z-error is recovered modulo the entire opposite stabilizer span. Symmetric CSS pairs, prescribed girth or degree conditions, and both-sector recovery remain separate, unassessed variants. A requirement that A carry a nontrivial Z logical is incompatible with correcting every supported Z-error: identity and that logical have the same syndrome. The concrete theorem `no_supported_Z_logical` checks this consequence for the present pair.

References

[1] K. D. Morris, T. Pllaha and C. A. Kelley, *Absorbing Sets in Quantum LDPC Codes*, arXiv:2307.14532v2, May 10, 2024, Sections II-B and VI, Definitions 3 and 5. `https://arxiv.org/html/2307.14532v2`. Journal metadata: *Adv. Math. Commun.* 23 (2026), 234–254, `https://doi.org/10.3934/amc.2026023`.

[2] *An Absorbing Set Without Gallager-B Trapping*, complete seven-variable proof and fourteen-coordinate CSS construction. `https://github.com/alectionary/vibe-math/tree/ff4172b090f4d38eadaf06fe419667f5b6bca585/formalization/gallager-b-absorbing`.

[3] R. W. Hamming, *Error detecting and error correcting codes*, Bell Syst. Tech. J. 29 (1950), 147–160. `https://doi.org/10.1002/j.1538-7305.1950.tb00463.x`.

[4] A. Steane, *Multiple particle interference and quantum error correction*, arXiv:quant-ph/9601029, January 29, 1996. https://arxiv.org/abs/quant-ph/9601029.

[5] D. Fattal, T. S. Cubitt, Y. Yamamoto, S. Bravyi and I. L. Chuang, *Entanglement in the stabilizer formalism*, arXiv:quant-ph/0406168, June 23, 2004. https://arxiv.org/abs/quant-ph/0406168.

[6] O. Makuta and R. Augusiak, *All genuinely entangled stabilizer subspaces are multipartite fully nonlocal*, arXiv:2312.08757, December 14, 2023; npj Quantum Information 11, 144 (2025), Lemma 1. https://doi.org/10.1038/s41534-025-01080-3.

A Checked proofs, exact interfaces and reproduction

The complete proof module imports only `Std`, using Lean 4.36.0-rc1. Finite statements use kernel-evaluated `decide`; the graph walks, spanning-tree propagation and eventual stopping arguments are structural proofs. The bit-vector projection simplifier `bv_normalize` produces ordinary proof terms. No native decision or SAT certificate oracle is used. Every theorem's audited dependencies are restricted to `propext`, `Classical.choice` and `Quot.sound`.

A.1 Public files and proof map

The complete current Lean module, immutable checked module, build and dependency manifest, and independent verifier are part of the published artifact. The source-comparison ledger records inspected versions and access limits; the complete source-screening ledger records harvested questions separately from this project-derived follow-up.

Written assertion	Matching formal declarations
Literal CSS construction	`xs`, `zs`, `css_commutation`, `two_CNOT_X_bridge`, `two_CNOT_Z_bridge`
Lemma 2: basis and ranks	`actual_x_rows_in_basis_span`, `basis_rows_are_actual`, `x_basis_independent`, `z_basis_independent`
Lemma 2: absorbing set	`selected_absorbing`, `induced_checks_even`, `selected_size`, `selected_variable_degrees`, `distinct_x_checks`
Logical witnesses	`logical_Z_commutes`, `logical_Z_not_stabilizer`, `logical_X_commutes`, `logical_pair_anticommutes`
Lemma 3: exact votes/output	`first_vote_counts`, `first_output_formula`, `outside_edges_zero`
Lemma 3: syndrome and recovery	`first_hard_matches`, `first_edge_matches`, `complete_quantum_recovery`
Stopped dynamics, Theorem 1	`terminal_matches`, `terminal_equals_first`, `all_checks_eventually_correct`, `all_variables_converge`, `selected_not_trapping`
Lemma 4	`hub_incidence`, `checks_nonempty`, `node_to_hub`, `decoding_graph_connected`
Lemma 5: every cut	`edge_0_projection` through `edge_13_projection`, `tree_crosses`, `cut_anticommutation`
Lemma 5: physical product obstruction	`span_x_commutes`, `CSSStabilizer`, `SplitsAcross`, `no_physical_product_cut`

The finite independence statements say that all binary coefficient words produce different combinations, while the two row/basis bridge statements identify the span of the actual redundant

rows. Together they are the binary rank certificates used in the written CSS dimension calculation. The supported-logical consequence is checked by `no_supported_Z_logical`.

A.2 Actual matrices and the complete decoding model

`Input = BitVec 7` includes every subset of A, including empty and full. `Word = BitVec 15` is the entire physical word, and `Msg` includes all actual check/variable pairs. Nonedges are masked out explicitly. `ZSpan` enumerates all combinations of the seven opposite generators; it does not check only individual rows. Unicode symbols in the following listing are rendered typographically; the linked UTF-8 Lean source is the checked file.

```
import Std

/-! A 15-coordinate connected CSS example. Only ‘Std‘ is imported.
All inputs are seven-bit supports on physical coordinates 0,...,6.
The actual decoding graph has 15 distinct X-check rows. -/
namespace ConnectedAbsorbing
set_option maxHeartbeats 0
set_option maxRecDepth 100000
abbrev Word := BitVec 15
abbrev Input := BitVec 7
abbrev V := Fin 15
abbrev ZC := Fin 7
abbrev Msg := V → V → Bool

def xs : List Word := [213,102,179,120,173,30,203,10880,13056,6528,15360,5760,3840,9600,16512]
def zs : List Word := [85,102,120,127,27265,13056,15360]
def xrow (c : V) : Word := xs[c.val]!
def zrow (d : ZC) : Word := zs[d.val]!
def vertices : List V := List.finRange 15

def bit (w : Word) (v : V) : Bool := w.getLsbD v.val
def parity (bs : List Bool) := bs.foldl Bool.xor false
def dot (a b : Word) : Bool := parity (vertices.map fun v => bit a v && bit b v)
def pack (f : V → Bool) : Word :=
  BitVec.ofNat 15 ((vertices.map fun v => if f v then 2^v.val else 0).foldl (·+·) 0)
def embed (e : Input) : Word := BitVec.ofNat 15 e.toNat
def support : Word := 127
def adj (c v : V) : Bool := bit (xrow c) v

def syndrome (e : Word) : Word := pack fun c => dot (xrow c) e

def majority (bs : List Bool) : Bool := decide (bs.length < 2 * bs.countP id)
def checkUpdate (s : Word) (x : Msg) : Msg := fun c v =>
  adj c v && Bool.xor (bit s c)
    (parity (vertices.map fun u => adj c u && (u != v) && x c u))
def hard (y : Msg) : Word :=
  pack fun v => majority ((vertices.filter fun c => adj c v).map fun c => y c v)
def variableUpdate (y : Msg) : Msg := fun c v =>
  adj c v && majority ((vertices.filter fun d => adj d v && (d != c)).map fun d => y d v)
def edgeSyndrome (x : Msg) : Word :=
  pack fun c => parity (vertices.map fun v => adj c v && x c v)
def firstChecks (e : Input) : Msg := checkUpdate (syndrome (embed e)) (fun _ _ => false)
def firstHard (e : Input) : Word := hard (firstChecks e)
def firstEdges (e : Input) : Msg := variableUpdate (firstChecks e)
def terminal (e : Input) : Word := if syndrome (embed e) = 0 then 0 else firstHard e

def Absorbing (a : Word) : Prop := ∀ v : V, bit a v = true →
  (vertices.filter fun c => adj c v && dot (xrow c) a).length <
  (vertices.filter fun c => adj c v && !(dot (xrow c) a)).length

theorem selected_absorbing : Absorbing support := by
  unfold Absorbing
  decide

theorem induced_checks_even : ∀ c : V, dot (xrow c) support = false := by decide
```

```
theorem selected_size : (vertices.filter fun v => bit support v).length = 7 := by decide

theorem distinct_x_checks : xs.Pairwise (fun a b => a ≠ b) := by decide

theorem css_commutation : ∀ c : V, ∀ d : ZC, dot (xrow c) (zrow d) = false := by decide

theorem first_hard_matches : ∀ e : Input,
  syndrome (firstHard e) = syndrome (embed e) := by decide

theorem first_edge_matches : ∀ e : Input,
  edgeSyndrome (firstEdges e) = syndrome (embed e) := by decide

theorem terminal_matches : ∀ e : Input,
  syndrome (terminal e) = syndrome (embed e) := by decide

theorem terminal_equals_first : ∀ e : Input, terminal e = firstHard e := by decide

def zCombination (a : BitVec 7) : Word :=
  ((List.finRange 7).map fun d => if a.getLsbD d.val then zrow d else 0).foldl (· ^^^ ·) 0

def xBasis : List Word := [213,102,120,10880,13056,15360,16512]
def xCombination (a : BitVec 7) : Word :=
  ((List.finRange 7).map fun d => if a.getLsbD d.val then xBasis[d.val]! else 0).foldl (· ^^^ ·) 0

def XSpan (w : Word) : Prop := ∃ a : BitVec 7, xCombination a = w
def ZSpan (w : Word) : Prop := ∃ a : BitVec 7, zCombination a = w

theorem actual_x_rows_in_basis_span : ∀ c : V, ∃ a : BitVec 7, xCombination a = xrow c := by decide

theorem basis_rows_are_actual : ∀ i : Fin 7, ∃ c : V, xBasis[i.val]! = xrow c := by decide

theorem complete_quantum_recovery : ∀ e : Input,
  ∃ a : BitVec 7, zCombination a = embed e ^^^ terminal e := by decide

theorem span_x_commutes : ∀ a : BitVec 7, ∀ d : ZC,
  dot (xCombination a) (zrow d) = false := by decide

theorem zero_z_span : ZSpan 0 := by
  unfold ZSpan
  decide

-- A genuine logical pair on the actual code, not on an unconnected spectator.
def logicalZ : Word := 32641
def logicalX : Word := 16256

theorem logical_Z_commutes : ∀ c : V, dot (xrow c) logicalZ = false := by decide

theorem logical_Z_not_stabilizer : ¬ ZSpan logicalZ := by
  unfold ZSpan
  decide

theorem logical_X_commutes : ∀ d : ZC, dot logicalX (zrow d) = false := by decide

theorem logical_pair_anticommutes : dot logicalX logicalZ = true := by decide
```

A.3 Constructed walks in the actual Tanner graph

The dependency chain is hub incidence → variable/check walks → walk reversal and concatenation → every ordered pair of nodes. Both sides of the bipartite graph are included as the two constructors of `Sum V V`.

```
-- Actual bipartite connectivity, including every check and every variable.
abbrev Node := Sum V V

def Edge : Node → Node → Prop
  | .inl v, .inr c => adj c v = true
  | .inr c, .inl v => adj c v = true
  | _, _ => False
```

```
inductive Walk : Node → Node → Prop
  | nil (u : Node) : Walk u u
  | cons {u v w : Node} : Edge u v → Walk v w → Walk u w

theorem edge_symm {u v : Node} (h : Edge u v) : Edge v u := by
  cases u <;> cases v <;> simp_all [Edge]

theorem walk_trans {u v w : Node} (h : Walk u v) (k : Walk v w) : Walk u w := by
  induction h with
  | nil => exact k
  | cons h _ ih => exact Walk.cons h (ih k)

theorem walk_reverse {u v : Node} (h : Walk u v) : Walk v u := by
  induction h with
  | nil u => exact Walk.nil u
  | @cons u v w h hv ih =>
    exact walk_trans ih (Walk.cons (edge_symm h) (Walk.nil u))

theorem hub_incidence : ∀ v : V, ∃ c : V,
  adj c v = true ∧ adj c 7 = true := by decide

theorem checks_nonempty : ∀ c : V, ∃ v : V, adj c v = true := by decide

theorem variable_to_hub (v : V) : Walk (.inl v) (.inl 7) := by
  obtain ⟨c,hv,h7⟩ := hub_incidence v
  exact Walk.cons (v := .inr c) hv (Walk.cons (v := .inl 7) h7 (Walk.nil (.inl 7)))

theorem node_to_hub (u : Node) : Walk u (.inl 7) := by
  cases u with
  | inl v => exact variable_to_hub v
  | inr c =>
    obtain ⟨v,hv⟩ := checks_nonempty c
    exact Walk.cons (v := .inl v) hv (variable_to_hub v)

theorem decoding_graph_connected (u v : Node) : Walk u v := by
  exact walk_trans (node_to_hub u) (walk_reverse (node_to_hub v))
```

Direct immutable link to the connectivity proof.

A.4 Projection identities and the universal cut proof

The fourteen two-coordinate identities are ground matrix calculations with a universally quantified cut. One complete representative proof is printed below; the thirteen parallel identities are in the complete linked module. `bv_normalize` reduces the bit operations; the remaining one-bit Boolean contradictions are proved by the two tiny kernel-decided lemmas. It introduces no trusted native computation dependency.

```
-- The two-row overlap certificate is a spanning tree on all15 coordinates.
-- For every nontrivial physical cut, at least one X/Z stabilizer pair has
-- anticommuting restrictions. A product stabilizer space cannot have this.
def treeX : List Nat := [0,2,4,6,9,11,13,14,2,5,5,9,12,12]
def treeZ : List Nat := [4,0,0,0,4,4,4,4,2,2,1,6,6,5]
def treeParent : List Nat := [0,0,0,0,7,7,7,7,4,4,2,11,11,9]
def treeChild : List Nat := [7,4,2,6,11,9,13,14,5,3,1,12,10,8]
def tx (i : Fin 14) : V := ⟨treeX[i.val]!, by decide +revert⟩
def tz (i : Fin 14) : ZC := ⟨treeZ[i.val]!, by decide +revert⟩

theorem cmp_impossible : ∀ a b : BitVec 1,
  (!(a == b) == (b == a)) = true → False := by decide

theorem bool_impossible : ∀ b : Bool, (!b == b) = true → False := by decide

theorem edge_0_projection (cut : Word) :
  dot (xrow 0 &&& cut) (zrow 4) =
    Bool.xor (cut.getLsbD 0) (cut.getLsbD 7) := by
  change dot ((213 : Word) &&& cut) (27265 : Word) = _
```

```
  simp [dot,vertices,parity,bit,List.finRange]
  bv_normalize
  all_goals first
    | exact bool_impossible _ (by assumption)
    | exact cmp_impossible _ _ (by assumption)
```

The following is the complete spanning-tree propagation and cut witness proof, followed by the whole product-obstruction proof. The disjunction in `TreeCrosses` uses exactly the fourteen edges of Table 1. If all comparisons are equal, every bit equals the root bit. Extensionality then makes the mask zero or all ones, contradicting a nontrivial cut.

```
def TreeCrosses (cut : Word) : Prop :=
  cut.getLsbD 0 ≠ cut.getLsbD 7 ∨
  cut.getLsbD 0 ≠ cut.getLsbD 4 ∨
  cut.getLsbD 0 ≠ cut.getLsbD 2 ∨
  cut.getLsbD 0 ≠ cut.getLsbD 6 ∨
  cut.getLsbD 7 ≠ cut.getLsbD 11 ∨
  cut.getLsbD 7 ≠ cut.getLsbD 9 ∨
  cut.getLsbD 7 ≠ cut.getLsbD 13 ∨
  cut.getLsbD 7 ≠ cut.getLsbD 14 ∨
  cut.getLsbD 4 ≠ cut.getLsbD 5 ∨
  cut.getLsbD 4 ≠ cut.getLsbD 3 ∨
  cut.getLsbD 2 ≠ cut.getLsbD 1 ∨
  cut.getLsbD 11 ≠ cut.getLsbD 12 ∨
  cut.getLsbD 11 ≠ cut.getLsbD 10 ∨
  cut.getLsbD 9 ≠ cut.getLsbD 8

theorem tree_crosses (cut : Word) (h0 : cut ≠ 0) (h1 : cut ≠ 32767) : TreeCrosses cut := by
  apply Classical.byContradiction
  intro hn
  simp only [TreeCrosses,not_or,Classical.not_not] at hn
  rcases hn with ⟨he0,he1,he2,he3,he4,he5,he6,he7,he8,he9,he10,he11,he12,he13⟩
  have hv0 : cut.getLsbD 0 = cut.getLsbD 0 := rfl
  have hv7 : cut.getLsbD 7 = cut.getLsbD 0 := he0.symm.trans hv0
  have hv4 : cut.getLsbD 4 = cut.getLsbD 0 := he1.symm.trans hv0
  have hv2 : cut.getLsbD 2 = cut.getLsbD 0 := he2.symm.trans hv0
  have hv6 : cut.getLsbD 6 = cut.getLsbD 0 := he3.symm.trans hv0
  have hv11 : cut.getLsbD 11 = cut.getLsbD 0 := he4.symm.trans hv7
  have hv9 : cut.getLsbD 9 = cut.getLsbD 0 := he5.symm.trans hv7
  have hv13 : cut.getLsbD 13 = cut.getLsbD 0 := he6.symm.trans hv7
  have hv14 : cut.getLsbD 14 = cut.getLsbD 0 := he7.symm.trans hv7
  have hv5 : cut.getLsbD 5 = cut.getLsbD 0 := he8.symm.trans hv4
  have hv3 : cut.getLsbD 3 = cut.getLsbD 0 := he9.symm.trans hv4
  have hv1 : cut.getLsbD 1 = cut.getLsbD 0 := he10.symm.trans hv2
  have hv12 : cut.getLsbD 12 = cut.getLsbD 0 := he11.symm.trans hv11
  have hv10 : cut.getLsbD 10 = cut.getLsbD 0 := he12.symm.trans hv11
  have hv8 : cut.getLsbD 8 = cut.getLsbD 0 := he13.symm.trans hv9
  have hall (i : Nat) (hi : i < 15) : cut.getLsbD i = cut.getLsbD 0 := by
    have hcases : i = 0 ∨ i = 1 ∨ i = 2 ∨ i = 3 ∨ i = 4 ∨ i = 5 ∨ i = 6 ∨ i = 7 ∨ i = 8 ∨ i = 9 ∨ i = 10
     ∨ i = 11 ∨ i = 12 ∨ i = 13 ∨ i = 14 := by omega
    rcases hcases with hi | hi | hi | hi | hi | hi | hi | hi | hi | hi | hi | hi | hi | hi | hi
    · simpa [hi] using hv0
    · simpa [hi] using hv1
    · simpa [hi] using hv2
    · simpa [hi] using hv3
    · simpa [hi] using hv4
    · simpa [hi] using hv5
    · simpa [hi] using hv6
    · simpa [hi] using hv7
    · simpa [hi] using hv8
    · simpa [hi] using hv9
    · simpa [hi] using hv10
    · simpa [hi] using hv11
    · simpa [hi] using hv12
    · simpa [hi] using hv13
    · simpa [hi] using hv14
  cases hroot : cut.getLsbD 0 with
  | false =>
    apply h0
```

```
    apply BitVec.eq_of_getLsbD_eq
    intro i hi
    rw [hall i hi,hroot]
    simp
  | true =>
    have hc : cut = BitVec.allOnes 15 := by
      apply BitVec.eq_of_getLsbD_eq
      intro i hi
      calc cut.getLsbD i = true := (hall i hi).trans hroot
           _ = (BitVec.allOnes 15).getLsbD i := by
             rw [BitVec.getLsbD_allOnes]
             simp only [hi, decide_true]
    have hf : BitVec.allOnes 15 = (32767 : Word) := by decide
    exact h1 (hc.trans hf)

theorem cut_anticommutation (cut : Word) (h0 : cut ≠ 0) (h1 : cut ≠ 32767) :
  ∃ i : Fin 14, dot (xrow (tx i) &&& cut) (zrow (tz i)) = true := by
  have h := tree_crosses cut h0 h1
  unfold TreeCrosses at h
  rcases h with h | h | h | h | h | h | h | h | h | h | h | h | h | h
  · refine ⟨0,?_⟩
    change dot (xrow 0 &&& cut) (zrow 4) = true
    rw [edge_0_projection]
    cases hu : cut.getLsbD 0 <;> cases hv : cut.getLsbD 7 <;> simp_all
  · refine ⟨1,?_⟩
    change dot (xrow 2 &&& cut) (zrow 0) = true
    rw [edge_1_projection]
    cases hu : cut.getLsbD 0 <;> cases hv : cut.getLsbD 4 <;> simp_all
  · refine ⟨2,?_⟩
    change dot (xrow 4 &&& cut) (zrow 0) = true
    rw [edge_2_projection]
    cases hu : cut.getLsbD 0 <;> cases hv : cut.getLsbD 2 <;> simp_all
  · refine ⟨3,?_⟩
    change dot (xrow 6 &&& cut) (zrow 0) = true
    rw [edge_3_projection]
    cases hu : cut.getLsbD 0 <;> cases hv : cut.getLsbD 6 <;> simp_all
  · refine ⟨4,?_⟩
    change dot (xrow 9 &&& cut) (zrow 4) = true
    rw [edge_4_projection]
    cases hu : cut.getLsbD 7 <;> cases hv : cut.getLsbD 11 <;> simp_all
  · refine ⟨5,?_⟩
    change dot (xrow 11 &&& cut) (zrow 4) = true
    rw [edge_5_projection]
    cases hu : cut.getLsbD 7 <;> cases hv : cut.getLsbD 9 <;> simp_all
  · refine ⟨6,?_⟩
    change dot (xrow 13 &&& cut) (zrow 4) = true
    rw [edge_6_projection]
    cases hu : cut.getLsbD 7 <;> cases hv : cut.getLsbD 13 <;> simp_all
  · refine ⟨7,?_⟩
    change dot (xrow 14 &&& cut) (zrow 4) = true
    rw [edge_7_projection]
    cases hu : cut.getLsbD 7 <;> cases hv : cut.getLsbD 14 <;> simp_all
  · refine ⟨8,?_⟩
    change dot (xrow 2 &&& cut) (zrow 2) = true
    rw [edge_8_projection]
    cases hu : cut.getLsbD 4 <;> cases hv : cut.getLsbD 5 <;> simp_all
  · refine ⟨9,?_⟩
    change dot (xrow 5 &&& cut) (zrow 2) = true
    rw [edge_9_projection]
    cases hu : cut.getLsbD 4 <;> cases hv : cut.getLsbD 3 <;> simp_all
  · refine ⟨10,?_⟩
    change dot (xrow 5 &&& cut) (zrow 1) = true
    rw [edge_10_projection]
    cases hu : cut.getLsbD 2 <;> cases hv : cut.getLsbD 1 <;> simp_all
  · refine ⟨11,?_⟩
    change dot (xrow 9 &&& cut) (zrow 6) = true
    rw [edge_11_projection]
    cases hu : cut.getLsbD 11 <;> cases hv : cut.getLsbD 12 <;> simp_all
  · refine ⟨12,?_⟩
    change dot (xrow 12 &&& cut) (zrow 6) = true
    rw [edge_12_projection]
```

```
    cases hu : cut.getLsbD 11 <;> cases hv : cut.getLsbD 10 <;> simp_all
  · refine ⟨13,?_⟩
    change dot (xrow 12 &&& cut) (zrow 5) = true
    rw [edge_13_projection]
    cases hu : cut.getLsbD 9 <;> cases hv : cut.getLsbD 8 <;> simp_all

def CSSStabilizer (p : Word × Word) : Prop := XSpan p.1 ∧ ZSpan p.2

def SplitsAcross (cut : Word) : Prop :=
  ∀ p : Word × Word, CSSStabilizer p → CSSStabilizer (p.1 &&& cut, p.2 &&& cut)

theorem no_physical_product_cut (cut : Word) (h0 : cut ≠ 0) (h1 : cut ≠ 32767) :
  ¬ SplitsAcross cut := by
  intro hs
  obtain ⟨i,hi⟩ := cut_anticommutation cut h0 h1
  obtain ⟨a,ha⟩ := actual_x_rows_in_basis_span (tx i)
  have hp : CSSStabilizer (xrow (tx i), 0) := ⟨⟨a,ha⟩,zero_z_span⟩
  obtain ⟨b,hb⟩ := (hs _ hp).1
  have hc := span_x_commutes b (tz i)
  rw [hb] at hc
  rw [hi] at hc
  contradiction
```

Direct immutable link to the physical product obstruction.

A.5 Stopped executions, gate bridges and analytic formulas

The source algorithm halts on a syndrome match. `stoppedOutput` therefore repeats the accepted terminal word after time one; the already proved first-step equalities make this a faithful representation for every input. The witnesses $L = 1$ prove both eventual quantifiers for all natural times, and their negations rule out a trapping input. The remainder prints the exact gate maps, the independent-basis interfaces, the supported-logical consequence and the analytic vote/output descriptions.

```
-- Eventual properties refer to the specified decoder with syndrome stopping.
def stoppedOutput (e : Input) (t : Nat) : Word := if t = 0 then 0 else terminal e

def EventuallyCorrect (e : Input) (c : V) : Prop :=
  ∃ L : Nat, ∀ t : Nat, L ≤ t →
    bit (syndrome (stoppedOutput e t)) c = bit (syndrome (embed e)) c

def EventuallyConverges (e : Input) (v : V) : Prop :=
  ∃ L : Nat, ∀ t : Nat, L ≤ t → bit (stoppedOutput e t) v = bit (stoppedOutput e (t+1)) v

def TrappingSelected : Prop := ∃ e : Input,
  (∃ c : V, ¬ EventuallyCorrect e c) ∨
  (∃ v : V, bit support v = true ∧ ¬ EventuallyConverges e v)

theorem all_checks_eventually_correct (e : Input) (c : V) : EventuallyCorrect e c := by
  refine ⟨1,?_⟩
  intro t ht
  have h : t ≠ 0 := by omega
  simp only [stoppedOutput,ite_eq_right h]
  rw [terminal_matches]

theorem all_variables_converge (e : Input) (v : V) : EventuallyConverges e v := by
  refine ⟨1,?_⟩
  intro t ht
  have h : t ≠ 0 := by omega
  have h' : t+1 ≠ 0 := by omega
  simp only [stoppedOutput,ite_eq_right h,ite_eq_right h']

theorem selected_not_trapping : ¬ TrappingSelected := by
  intro ⟨e,h⟩
  rcases h with ⟨c,hc⟩ | ⟨v,hv,hf⟩
  · exact hc (all_checks_eventually_correct e c)
  · exact hf (all_variables_converge e v)
```

```
-- Explicit finite construction bridge to two CNOTs on the earlier CSS pair
-- plus one X-stabilized auxiliary qubit. The final code has no physical factor.
def originalX : List Word := [85,102,51,120,45,30,75,10880,13056,6528,15360,5760,3840,9600,16384]
def originalZ : List Word := [85,102,120,127,10880,13056,15360]
def cnotX (c t : V) (w : Word) : Word :=
  if bit w c then w ^^^ BitVec.ofNat 15 (2^t.val) else w
def cnotZ (c t : V) (w : Word) : Word :=
  if bit w t then w ^^^ BitVec.ofNat 15 (2^c.val) else w

theorem two_CNOT_X_bridge : originalX.map (fun w => cnotX 14 7 (cnotX 0 7 w)) = xs := by decide
theorem two_CNOT_Z_bridge : originalZ.map (fun w => cnotZ 14 7 (cnotZ 0 7 w)) = zs := by decide

theorem x_basis_independent : ∀ a b : BitVec 7,
  xCombination a = xCombination b ↔ a = b := by decide

theorem z_basis_independent : ∀ a b : BitVec 7,
  zCombination a = zCombination b ↔ a = b := by decide

theorem no_supported_Z_logical (e : Input) (h : syndrome (embed e) = 0) : ZSpan (embed e) := by
  obtain ⟨a,ha⟩ := complete_quantum_recovery e
  have ht : terminal e = 0 := by simp [terminal,h]
  refine ⟨a,?_⟩
  simpa [ht] using ha

-- Analytic descriptions, checked against the literal fifteen-coordinate model.
def label (e : Input) : BitVec 3 :=
  ((List.finRange 7).map fun i => if e.getLsbD i.val then BitVec.ofNat 3 (i.val+1) else 0).foldl (· ^^^ ·) 0

def singletonOutput (p : BitVec 3) : Word :=
  if p = 0 then 0 else BitVec.ofNat 15 (2^(p.toNat-1))

def initialOnes (e : Input) (v : V) : Nat :=
  ((vertices.filter fun c => adj c v).map fun c => firstChecks e c v).countP id

theorem selected_variable_degrees : ∀ v : V, bit support v = true →
  (vertices.filter fun c => adj c v).length = 4 := by decide

theorem first_output_formula : ∀ e : Input, firstHard e = singletonOutput (label e) := by decide

theorem first_vote_counts : ∀ e : Input, ∀ i : Fin 7,
  initialOnes e ⟨i.val,by omega⟩ =
    if label e = 0 then 0 else if (label e).toNat = i.val+1 then 4 else 2 := by decide

theorem outside_edges_zero : ∀ e : Input, ∀ c v : V,
  7 ≤ v.val → firstEdges e c v = false := by decide
```

Direct immutable link to recovery modulo the full opposite span.

A.6 Reproduction and evidence levels

Run `python3 build.py -lean /path/to/lean` in the artifact folder with the pinned toolchain, then `python3 verify.py -check`. The clean build reports every theorem's dependencies. The independent literal-list implementation reconstructs the two CNOT maps, all 128 inputs, complete binary spans and all $32,766$ nontrivial cuts. Its JSON certificate supplies diagnostics, while Lean supplies the formal proofs. The original fourteen-coordinate artifact remains preserved separately.